Relación Ejercicios Tema 2 Kotlin

Ejercicio 1. Sistema de validación de usuarios

Desarrolla un sistema que reciba el nombre, correo electrónico, contraseña y edad de un usuario (esta última como `String`). Crea funciones independientes para limpiar el nombre, validar el correo, comprobar la contraseña y convertir la edad de forma segura.

La contraseña debe tener al menos ocho caracteres, una letra mayúscula y un número. El usuario debe ser mayor de edad.

Funciones de Kotlin recomendadas: `trim()`, `lowercase()`, `contains()`, `any()`, `isUpperCase()`, `isDigit()` y `toIntOrNull()`.

Ejercicio 2. Carrito de compra con descuentos

Crea una clase `Producto` con nombre, precio y cantidad. Construye una lista con al menos cinco productos. Implementa funciones para calcular el subtotal de cada producto, el total del carrito, aplicar un descuento del 15 % cuando se superen los 100 € y encontrar el producto más caro. No se admitirán precios ni cantidades negativas.

Funciones de Kotlin recomendadas: `sumOf()`, `maxByOrNull()`, `filter()` y `coerceAtLeast()`.

Ejercicio 3. Gestor de calificaciones de alumnos

Crea una clase `Alumno` con nombre y nota. Registra al menos ocho alumnos. Implementa funciones para calcular la nota media, separar aprobados y suspensos, ordenar los alumnos por nota descendente y localizar al alumno con la nota más alta. Añade una función `buscarAlumno(nombre)` que permita buscar sin distinguir mayúsculas y minúsculas.

Funciones de Kotlin recomendadas: `average()`, `filter()`, `sortedByDescending()`, `maxByOrNull()`, `find()` y `equals(ignoreCase = true)`.

Ejercicio 4. Gestor de tareas con prioridades

Crea una clase `Tarea` con título, prioridad (1–3) y estado de finalización. Implementa funciones para añadir tareas, marcarlas como completadas, mostrar únicamente las pendientes y ordenar las tareas pendientes por prioridad. Añade una función que calcule el porcentaje de tareas completadas, teniendo en cuenta que la lista podría estar vacía.

Funciones de Kotlin recomendadas: `filter()`, `count()`, `sortedByDescending()`, `find()` e `isEmpty()`.

Ejercicio 5. Buscador inteligente de contactos

Crea una clase `Contacto` con nombre, teléfono y correo electrónico. Almacena diez contactos. Desarrolla funciones para buscar por una parte del nombre, filtrar contactos por dominio de correo y eliminar contactos duplicados según su número de teléfono. El buscador debe ignorar mayúsculas, minúsculas y espacios sobrantes.

Funciones de Kotlin recomendadas: `contains(ignoreCase = true)`, `trim()`, `filter()`, `distinctBy()` y `substringAfter()`.

Ejercicio 6. Analizador de mensajes de texto

Desarrolla una herramienta que analice un mensaje introducido por el usuario. Debe contar caracteres sin espacios, contar palabras, obtener la palabra más larga y detectar si contiene alguna palabra prohibida de una lista. Crea una función adicional que devuelva el mensaje limpio, eliminando espacios al principio y al final y sustituyendo secuencias de varios espacios por uno solo.

Funciones de Kotlin recomendadas: `trim()`, `split()`, `filter()`, `maxByOrNull()`, `any()` y `replace(Regex(...))`.

Ejercicio 7. Simulador de cuenta bancaria

Crea una clase `CuentaBancaria` con titular, saldo privado y una lista de movimientos. Implementa métodos para ingresar, retirar, consultar el saldo y obtener el historial. Las retiradas no pueden superar el saldo y las cantidades deben ser positivas. Añade funciones que calculen el total ingresado y retirado a partir de los movimientos registrados.

Funciones de Kotlin recomendadas: `filter()`, `sumOf()` y `add()`. Se deberá utilizar encapsulación mediante `private`.

Ejercicio 8. Sistema de reservas de actividades

Crea una clase `Reserva` con nombre de usuario, actividad, número de plazas y estado. Implementa funciones para registrar reservas, calcular las plazas ocupadas por actividad, filtrar reservas activas y cancelar una reserva por usuario y actividad. La aplicación debe impedir que una reserva supere las plazas disponibles. Utiliza un mapa para almacenar el aforo de cada actividad.

Funciones de Kotlin recomendadas: `filter()`, `sumOf()`, `groupBy()`, `find()` y operaciones sobre `Map`.

Ejercicio 9. Analizador de temperaturas

Una aplicación meteorológica almacena las temperaturas de siete días en una lista. Crea funciones para calcular la temperatura media, máxima y mínima, contar los días por encima de la media y convertir las temperaturas de Celsius a Fahrenheit. Añade una función que clasifique cada día como frío, templado o caluroso según unos límites establecidos por ti.

Funciones de Kotlin recomendadas: `average()`, `maxOrNull()`, `minOrNull()`, `count()`, `map()` y `when`.

Ejercicio 10. Miniaplicación de gestión de biblioteca

Crea una clase `Libro` con identificador, título, autor, categoría y disponibilidad. Desarrolla una clase `Biblioteca` que almacene los libros y permita registrarlos, buscarlos por título o autor, prestarlos y devolverlos. Añade funciones para mostrar libros disponibles, ordenar por título y obtener cuántos libros existen por categoría.

El sistema debe impedir prestar un libro que ya está prestado y registrar dos libros con el mismo identificador.

Funciones de Kotlin recomendadas: `find()`, `any()`, `filter()`, `sortedBy()` y `groupBy()`.

Recomendaciones generales para el alumnado

- Cada ejercicio debe incluir una función `main()` para comprobar su funcionamiento y al menos dos funciones propias, además de `main()`, salvo que la estructura del ejercicio requiera más.
- Se deben utilizar parámetros y valores de retorno (`return`) en las funciones.
- Se recomienda utilizar las funciones predefinidas de Kotlin indicadas en cada ejercicio, evitando resolverlo todo con bucles `for`.
- Los ejercicios que requieran clases deberán utilizar constructores, propiedades y métodos.
- El código deberá incluir comentarios `//` o `/* */` explicando las operaciones más importantes.
- Se deberán probar casos correctos e incorrectos, comprobando que el programa responde adecuadamente.
- La función `main()` debe utilizarse principalmente para crear datos de prueba, llamar a las funciones y mostrar resultados mediante `println()`.